

AI-SA-IN-CD-000001

Module 8.1 · LLM Evaluation Fundamentals · AI Safety: Evaluation, Guardrails & Responsible Deployment

Building Evaluation Pipelines

lm-eval-harness · custom tasks · LLM-as-judge integration · W&B logging · GitHub Actions CI

25 min

Duration

B3 – Apply

Bloom's

L3 – Advanced

Level

Coding Lab

Domain

Learning Objectives

- 1 Set up a reproducible Python evaluation environment and project directory structure
- 2 Run MMLU and TruthfulQA using lm-evaluation-harness with YAML task configs
- 3 Implement a custom held-out evaluation task with JSONL dataset and metric aggregation
- 4 Build an LLM-as-judge client with position-bias mitigation and structured output parsing
- 5 Log evaluation results to Weights & Biases with artifact versioning and run comparison
- 6 Configure a GitHub Actions CI workflow that automatically blocks regression on key metrics

PART A · 5 slides

Environment Setup & Project Architecture

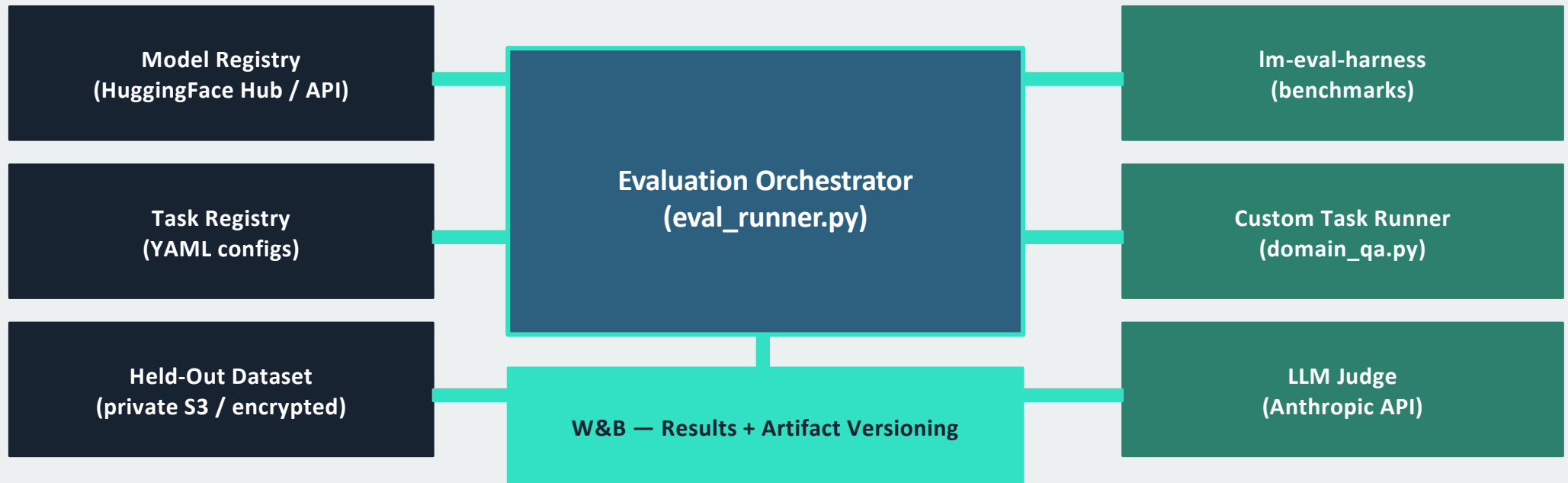
Evaluation Pipeline Architecture — Overview

Evaluation Pipeline

A reproducible, automated system that: (1) loads one or more models, (2) runs them on standardised benchmarks and custom held-out tasks, (3) scores outputs with automated metrics or an LLM judge, and (4) logs all results to a tracking system for comparison and CI/CD regression testing.

- Reproducibility: same pipeline + same model → identical results across machines and time
- Composability: benchmark tasks, judge calls, metric aggregation, and logging are independent modules
- Scalability: must handle 50 held-out examples (dev) to 15,000 benchmark questions (prod) on the same code path
- Separation: evaluation code must never import training code — use separate conda environments

Evaluation Pipeline — Data & Control Flow



GitHub Actions CI triggers this entire pipeline on every push to main.

Environment Setup — Installation

bash

```
# Create isolated evaluation environment
conda create -n llm_eval python=3.11 -y && conda activate llm_eval

# Core evaluation framework
pip install lm-eval==0.4.3           # EleutherAI eval harness
pip install anthropic==0.28.0       # Anthropic judge client
pip install openai==1.35.0          # OpenAI API client
pip install wandb==0.17.0           # Experiment tracking

# NLP utilities
pip install bert-score==0.3.13 rouge-score==0.1.2
pip install torch==2.3.0 transformers==4.42.0

# Pin versions for reproducibility
pip freeze > requirements-eval.txt
```

Never mix training and evaluation environments — separate conda envs prevent accidental import of gradient checkpointing or distributed training code.

Project Directory Structure

bash

```
llm_eval_pipeline/
├── eval/
│   ├── configs/                # lm-eval YAML task configs
│   │   ├── mmlu_medical.yaml
│   │   └── custom_qa.yaml
│   ├── custom_tasks/
│   │   ├── __init__.py
│   │   └── domain_qa.py        # Custom evaluator class
│   ├── judge/
│   │   ├── judge_client.py     # LLM judge wrapper
│   │   └── prompts/pairwise.txt
│   └── eval_runner.py          # Main orchestration script
├── results/                   # Auto-generated; gitignored
├── ci/eval_ci.yaml            # GitHub Actions workflow
├── requirements-eval.txt
└── README.md
```

Environment Dependency Map

lm-eval-harness
0.4.3

transformers
4.42.0

torch 2.3.0
(CUDA 12.1)

anthropic
0.28.0

wandb
0.17.0

bert-score
0.3.13

Benchmark Tasks
(YAML + HF datasets)

Custom Evaluator
(domain_qa.py)

GitHub Actions
CI Runner

Row 1: Core evaluation framework · Row 2: External integrations · Row 3: Our custom pipeline components

PART B · 8 slides

Running Standard Benchmarks with Im-evaluation-harness

lm-evaluation-harness — Framework Overview

lm-evaluation-harness (EleutherAI)

A unified evaluation framework supporting 200+ benchmark tasks across model providers (HuggingFace, OpenAI, Anthropic, together.ai). Tasks are defined as YAML configs and Python classes enabling reproducible, standardised benchmark evaluation with a single CLI command.

- Supports: multiple-choice, text generation, code generation, and log-likelihood evaluation modes
- Built-in: MMLU (57 subjects), TruthfulQA, GSM8K, HellaSwag, ARC, WinoGrande, HumanEval, and 200+ more
- Few-shot control: `--num_fewshot` parameter; each task has a research-validated default
- Output: JSON results with per-task aggregate scores, stderr, and full sample-level predictions

Im-eval YAML Config — Medical MMLU Subset

YAML

```
# eval/configs/mmlu_medical.yaml
group: mmlu_medical
group_alias: MMLU Medical Domains

task:
  - mmlu_clinical_knowledge
  - mmlu_medical_genetics
  - mmlu_anatomy
  - mmlu_professional_medicine
  - mmlu_college_biology
  - mmlu_college_medicine

metadata:
  version: 1
  description: Medical-domain MMLU subset
  num_fewshot: 5
  metric_list:
    - metric: acc_norm
```

YAML task groups aggregate multiple subtasks into one evaluable unit — use for domain-specific evaluation suites in healthcare, legal, or scientific AI products.

lm-eval — HuggingFace Model Loading

Python

```
from lm_eval.models.huggingface import HFLM
from lm_eval import simple_evaluate

lm = HFLM(
    pretrained='meta-llama/Meta-Llama-3.1-8B-Instruct',
    dtype='bfloat16',
    device='cuda',
    batch_size=16,          # Tune to GPU VRAM
)

results = simple_evaluate(
    model=lm,
    tasks=['mmlu_medical'],
    num_fewshot=5,
    batch_size=16,
    write_out=True,         # Save per-sample predictions
    output_path='results/mmlu_medical_llama3.8b.json',
    random_seed=42,         # Reproducibility
)
```

Set random_seed=42 for reproducibility. Use batch_size='auto' to maximise GPU utilisation automatically.

lm-eval CLI — Running MMLU (All 57 Subjects)

bash

```
lm_eval \
  --model hf \
  --model_args pretrained=meta-llama/Meta-Llama-3.1-8B-Instruct,dtype=bfloat16 \
  --tasks mmlu \
  --num_fewshot 5 \
  --batch_size 16 \
  --output_path results/mmlu_llama31_8b/ \
  --log_samples \
  --seed 42
```

Expected output (abbreviated):

hf (pretrained=...), num_fewshot: 5, batch_size: 16

#	Task	Filter	n-shot	Metric		Value	Stderr
#	-----	-----	-----	-----	---	-----	-----
#	mmlu	none	5	acc_norm	↑	0.683	± 0.004

Approximate runtime: 8B model on full MMLU (~15K questions) = 45 min on A100 80GB with batch_size=16.

lm-eval CLI — Running TruthfulQA (0-shot)

bash

```
# TruthfulQA MC2 (multiple correct answers per question)
lm_eval \
  --model hf \
  --model_args pretrained=meta-llama/Meta-Llama-3.1-8B-Instruct,dtype=bfloat16 \
  --tasks truthfulqa_mc2 \
  --num_fewshot 0 \
  --output_path results/tqa_llama31_8b/ \
  --seed 42

# Parse results
python -c "
import json
r = json.load(open('results/tqa_llama31_8b/results.json'))
print(r['results']['truthfulqa_mc2']['acc',None]) # e.g. 0.621
"
```

TruthfulQA MC2 measures the model's ability to assign higher probability to true answers versus false ones. A random model scores ~50%; human baseline is ~94%.

Im-eval Built-In Tasks — Quick Reference

Task Name	Benchmark	Metric	Shots	A100 Runtime (8B)
mmlu	MMLU	acc_norm	5	~45 min
truthfulqa_mc2	TruthfulQA	acc	0	~3 min
gsm8k	GSM8K	exact_match	8 (CoT)	~20 min
hellaswag	HellaSwag	acc_norm	10	~15 min
arc_challenge	ARC-Challenge	acc_norm	25	~8 min
winogrande	WinoGrande	acc	5	~5 min
humaneval	HumanEval	pass@1	0	~30 min

Parsing lm-eval Output — Results JSON

Python

```
import json

with open('results/mmlu_llama31_8b/results.json') as f:
    r = json.load(f)

# Aggregate MMLU score
agg = r['results']['mmlu']['acc_norm,none']
print(f'MMLU (5-shot, acc_norm): {agg:.3f}')    # 0.683

# Per-subject breakdown
for task, metrics in r['results'].items():
    if task.startswith('mmlu_') and task != 'mmlu':
        acc = metrics.get('acc_norm,none', 0)
        print(f'{task:<40} acc={acc:.3f}')

# Save structured summary for W&B logging
summary = {k: v.get('acc_norm,none', v.get('acc,none'))
            for k, v in r['results'].items() }
```

Always save the full results.json as a W&B artifact — it contains per-sample predictions essential for post-hoc error analysis.

Benchmark Results — JSON Schema

```
results.json
├── config: {model, tasks, num_fewshot, batch_size, seed, git_hash}
├── date: 2026-05-15T14:32:00Z
├── results:
│   ├── mmlu: {acc_norm,none: 0.683, acc_norm_stderr,none: 0.004}
│   └── mmlu_clinical_knowledge: {acc_norm,none: 0.721, ...}
├── samples: (only with --log_samples)
│   └── [{doc_id, query, target, prediction, correct, logprob}, ...]
```

Store results.json as a W&B artifact tied to the model checkpoint hash. Never overwrite — always append versioned runs.

PART C · 7 slides

Building Custom Held-Out Evaluation Tasks

Custom Held-Out Evaluation Task — Design

Custom Held-Out Evaluation Task

A privately held, curated evaluation dataset specific to your deployment domain. Unlike public benchmarks, it cannot be contaminated via pre-training, directly measures capability on your actual use case, and can be updated as the task distribution evolves.

- Dataset format: JSONL — one JSON object per line; fields: id, question, answer, category, difficulty
- Minimum viable set: 200 examples, ≥ 5 categories, ≥ 3 difficulty levels
- Storage: private S3 bucket with server-side encryption; never commit plaintext to a public repository
- Metrics: `exact_match` (factual QA) + F1 (partial credit) + BERTScore (semantic) + LLM-judge (holistic)

Custom Dataset — JSONL Builder

Python

```
# eval/custom_tasks/build_dataset.py
import json, random

def build_held_out_jsonl(qa_pairs: list[dict], out: str):
    """Write QA pairs to JSONL.
    Required fields: id, question, answer, category, difficulty"""
    required = {'id', 'question', 'answer', 'category', 'difficulty'}
    for item in qa_pairs:
        assert required <= item.keys(), f'Missing fields in {item["id"]}'
    random.shuffle(qa_pairs) # Remove ordering bias
    with open(out, 'w') as f:
        for item in qa_pairs:
            f.write(json.dumps(item) + '\n')
    print(f'Wrote {len(qa_pairs)} examples → {out}')

# Usage
build_held_out_jsonl(curated_pairs, 'eval/data/held_out_qa.jsonl')
```

Shuffle before writing to prevent positional bias in few-shot selection. Assert required fields to catch data quality issues early.

Custom Evaluator Class — domain_qa.py

Python

```
# eval/custom_tasks/domain_qa.py
from lm_eval.api.task import ConfigurableTask
from lm_eval.api.metrics import mean, f1_score

class DomainQATask(ConfigurableTask):
    VERSION = 1
    DATASET_PATH = 'eval/data/held_out_qa.jsonl'

    def doc_to_text(self, doc):
        return f"Question: {doc['question']}\nAnswer:"

    def doc_to_target(self, doc):
        return doc['answer']

    def process_results(self, doc, results):
        pred = results[0].strip()          # Model's generated answer
        ref = doc['answer']
        return {'exact_match': pred.lower() == ref.lower(),
                'f1': f1_score(pred, ref)}

    def aggregation(self):
        return {'exact_match': mean, 'f1': mean}
```

Metric Aggregation — BERTScore + Category Breakdown

Python

```
from bert_score import score as bert_score
import json, statistics

def aggregate_results(predictions_path: str) -> dict:
    with open(predictions_path) as f:
        samples = [json.loads(l) for l in f]

    preds = [s['prediction'] for s in samples]
    refs = [s['answer'] for s in samples]

    P, R, F1 = bert_score(preds, refs, lang='en', verbose=False)
    bert_f1 = F1.mean().item()

    # Per-category breakdown
    by_cat = {}
    for s in samples:
        cat = s.get('category', 'unknown')
        by_cat.setdefault(cat, []).append(s['exact_match'])
    cat_accs = {cat: statistics.mean(vals) for cat, vals in by_cat.items()}

    return {'bertscore_f1': bert_f1, 'by_category': cat_accs}
```

Category-level breakdown is critical for identifying domain gaps — an aggregate 75% accuracy may hide 40% performance on a safety-critical category.

Custom Held-Out Task — Example Results Table

Category	N	exact_match	F1	BERTScore F1	Judge Score /10
Clinical Protocols	40	0.725	0.781	0.853	7.4
Drug Interactions	35	0.657	0.712	0.831	6.8
Diagnostic Criteria	45	0.778	0.826	0.877	7.9
Patient Safety	40	0.700	0.755	0.842	7.1
Research Summaries	40	0.625	0.698	0.812	6.5
OVERALL	200	0.699	0.754	0.843	7.1

Custom Task — YAML Configuration

YAML

```
# eval/configs/custom_qa.yaml
task: domain_qa_held_out
task_alias: Domain QA Held-Out Set

dataset_path: eval/data/held_out_qa.jsonl
dataset_kwargs:
  split: test

doc_to_text: "Question: {{question}}\nAnswer:"
doc_to_target: "{{answer}}"

generation_kwargs:
  max_gen_toks: 256
  temperature: 0.0          # Deterministic for reproducibility
  until: ["\n\n"]          # Stop on double newline
```

`metric_list:`
- `metric: exact_match`
- `metric: f1`
`temperature=0.0` is mandatory for reproducible evaluation. Greedy decoding ensures the same model always produces the same output.

PART D · 9 slides

LLM-as-Judge Integration

LLM-as-Judge Integration — Design Goals

judge_client.py Design Goals

A production-ready LLM judge client that: (1) calls the judge API with a structured prompt, (2) mitigates position bias via response-order randomisation, (3) parses structured output reliably, and (4) returns results in a JSON-serialisable format for W&B logging.

- Bias mitigation: evaluate every (prompt, A, B) pair twice — once with original order, once with swapped order
- Structured output: prompt the judge to return a JSON object with keys: verdict, score, reasoning
- Retry logic: wrap API calls with exponential backoff and a maximum of 3 retries
- Async: use asyncio for concurrent judge calls — reduces wall-clock time by 4–8×

judge_client.py — Core Implementation

Python

```
import anthropic, json, random

client = anthropic.Anthropic() # Reads ANTHROPIC_API_KEY from env

def judge_pairwise(prompt: str, resp_a: str, resp_b: str,
                  judge_model: str = 'claude-sonnet-4-6') -> dict:
    # Mitigate position bias: randomise order
    order = random.random() > 0.5
    first = resp_a if order else resp_b
    second = resp_b if order else resp_a
    first_label = 'Response A' if order else 'Response B'
    second_label = 'Response B' if order else 'Response A'

    msg = build_judge_message(prompt, first, second, first_label, second_label)
    response = client.messages.create(
        model=judge_model, max_tokens=512,
        messages=[{'role': 'user', 'content': msg}]
    )
    return parse_verdict(response.content[0].text, order)
```

Judge Prompt Template

Python

```
JUDGE_PROMPT = """You are an expert AI evaluator. Compare two responses.
Return ONLY a JSON object with keys: verdict, score_a, score_b, reasoning.

Scoring (1-10): 1=completely wrong, 5=adequate, 10=excellent.
Verdict: 'Response A', 'Response B', or 'Tie'.

Question: {question}

Response A: {response_a}

Response B: {response_b}

Return ONLY valid JSON. No preamble or markdown code blocks."""

def build_judge_message(q, a, b, la, lb):
    return JUDGE_PROMPT.format(question=q, response_a=a, response_b=b)
```

Instruct the judge to return ONLY JSON. Strip markdown fences with `response.strip().lstrip('```json').rstrip('```')` before `json.loads()`.

Parsing Judge Output — Structured Verdict

Python

```
def parse_verdict(raw: str, order_is_original: bool) -> dict:
    """Parse JSON verdict; remap labels if order was swapped."""
    clean = raw.strip().lstrip('```json').lstrip('```').rstrip('```')
    try:
        data = json.loads(clean)
    except json.JSONDecodeError:
        return {'verdict': 'parse_error', 'score_a': 0, 'score_b': 0}

    verdict = data.get('verdict', 'Tie')
    # Remap if order was swapped
    if not order_is_original:
        if verdict == 'Response A': verdict = 'Response B'
        elif verdict == 'Response B': verdict = 'Response A'

    return {
        'verdict': verdict,
        'score_a': data.get('score_a', 5),
        'score_b': data.get('score_b', 5),
        'reasoning': data.get('reasoning', '')
    }
```

Always handle `json.JSONDecodeError` — LLM judges occasionally include preamble text despite explicit instructions to return only JSON.

Batch LLM-Judge Evaluation — Async Runner

Python

```
import asyncio
from judge_client import judge_pairwise

async def judge_batch_async(eval_pairs: list[dict]) -> list[dict]:
    """Run judge on list of {prompt, response_a, response_b} dicts."""
    sem = asyncio.Semaphore(8)      # Max 8 concurrent API calls

    async def judge_one(pair):
        async with sem:
            loop = asyncio.get_event_loop()
            result = await loop.run_in_executor(
                None, judge_pairwise,
                pair['prompt'], pair['response_a'], pair['response_b']
            )
            return {**pair, 'judge_result': result}

    return await asyncio.gather(*[judge_one(p) for p in eval_pairs])

# Usage
results = asyncio.run(judge_batch_async(eval_pairs))
```

Semaphore limits concurrent API calls to prevent rate-limit errors. Adjust to match your API tier's requests-per-minute limit.

Aggregating Judge Results — Win Rate Computation

Python

```
from collections import Counter

def compute_win_rates(judge_results: list[dict]) -> dict:
    verdicts = [r['judge_result']['verdict'] for r in judge_results]
    counts = Counter(verdicts)
    total = len(verdicts)

    win_a = counts.get('Response A', 0) / total
    win_b = counts.get('Response B', 0) / total
    tie = counts.get('Tie', 0) / total
    errors = counts.get('parse_error', 0) / total

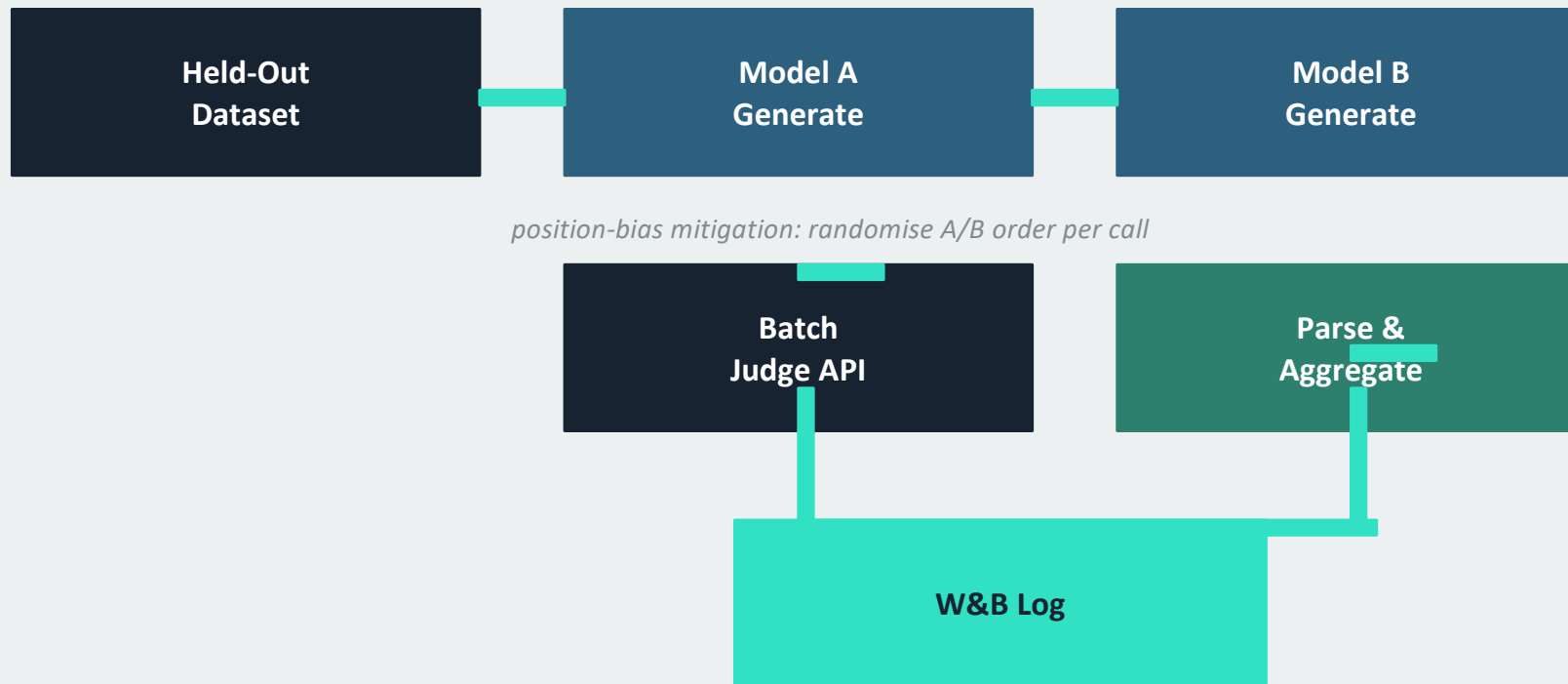
    return {
        'win_rate_model_a': round(win_a, 3),
        'win_rate_model_b': round(win_b, 3),
        'tie_rate': round(tie, 3),
        'parse_error_rate': round(errors, 3),
        'n_evaluations': total
    }
```

`parse_error_rate > 5%` signals a prompt design problem — the judge is not following output format instructions reliably.

LLM Judge Results — Example Output Table

Model A	Model B	Win A%	Tie%	Win B%	Avg Score A	Avg Score B
LLaMA-3.1-8B-Instruct	LLaMA-3.1-8B-Base	71.2%	8.5%	20.3%	7.6	5.8
LLaMA-3.1-8B-Instruct	Mistral-7B-Instruct	54.1%	12.3%	33.6%	7.4	7.1
LLaMA-3.1-8B-Instruct	Gemma-2-9B-IT	44.8%	15.2%	40.0%	7.3	7.5

Judge Integration — Full Evaluation Flow



PART E · 12 slides

W&B Logging & GitHub Actions CI

Weights & Biases — Evaluation Tracking

W&B Evaluation Tracking

Weights & Biases (W&B) provides experiment tracking, artifact versioning, and visualisation for evaluation runs. Every evaluation run is stored as a versioned W&B run with: scalar metrics, per-sample tables, model checkpoint references, and comparison dashboards across runs.

- Run metadata: model name, checkpoint hash, task config hash, evaluation date, and git commit
- Metric tables: per-sample predictions logged as `wandb.Table` — enables fine-grained failure analysis
- Artifact versioning: `results.json` and model checkpoint linked as W&B artifacts — fully reproducible
- Run groups: group all evaluations of the same model variant under a W&B group for easy comparison

W&B — Initialise Evaluation Run

Python

```
import wandb, json, os

def init_eval_run(model_name: str, task: str, config: dict) -> wandb.run:
    run = wandb.init(
        project='llm-eval-pipeline',
        group=model_name,          # Group runs by model
        job_type='evaluation',
        name=f'{model_name}_{task}_{config["num_fewshot"]}shot',
        config={
            'model':      model_name,
            'task':       task,
            'num_fewshot': config['num_fewshot'],
            'batch_size': config['batch_size'],
            'seed':       config.get('seed', 42),
            'git_hash':   os.popen('git rev-parse HEAD').read().strip(),
        }
    )
    return run
```

Always log `git_hash` — it makes every W&B run trivially reproducible by checking out that commit and re-running the pipeline.

W&B — Logging Aggregate Metrics

Python

```
def log_aggregate_metrics(run, benchmark_results: dict, judge_results: dict):
    # Log scalar metrics
    run.log({
        'mmlu/acc_norm': benchmark_results['mmlu']['acc_norm,none'],
        'truthfulqa/acc': benchmark_results.get('truthfulqa_mc2', {}).get('acc,none', 0),
        'custom_qa/exact_match': benchmark_results.get('domain_qa', {}).get('exact_match', 0),
        'custom_qa/bertscore_f1': benchmark_results.get('domain_qa', {}).get('bertscore_f1', 0),
        'judge/win_rate': judge_results['win_rate_model_a'],
        'judge/avg_score': judge_results.get('avg_score_a', 0),
    })

    # Log summary badges (appear in W&B run table)
    run.summary['mmlu_acc_norm'] = benchmark_results['mmlu']['acc_norm,none']
    run.summary['judge_win_rate'] = judge_results['win_rate_model_a']
```

`run.summary` values appear in the W&B run comparison table — use them for the key metrics you always want visible at a glance.

W&B — Logging Per-Sample Predictions Table

Python

```
def log_predictions_table(run, samples: list[dict]):
    columns = ['id', 'category', 'difficulty',
               'question', 'reference', 'prediction',
               'exact_match', 'f1', 'judge_score']

    table = wandb.Table(columns=columns)
    for s in samples:
        table.add_data(
            s['id'], s['category'], s['difficulty'],
            s['question'][:200],      # Truncate for display
            s['answer'][:200],
            s['prediction'][:200],
            s['exact_match'],
            round(s['f1'], 3),
            s.get('judge_score', -1)
        )

    run.log({'predictions': table})
    print(f'Logged {len(samples)} predictions to W&B table')
```

W&B Tables support filtering, sorting, and correlation analysis in the browser — essential for identifying failure patterns.

W&B — Artifact Versioning

Python

```
def log_results_artifact(run, results_path: str, model_name: str):
    artifact = wandb.Artifact(
        name=f'eval-results-{model_name}',
        type='evaluation',
        description='lm-eval results.json + judge output',
        metadata={'model': model_name, 'task': 'mmlu+custom_qa'}
    )
    artifact.add_file(results_path)          # results.json
    artifact.add_file('results/judge_out.json') # Judge verdicts
    run.log_artifact(artifact)

    # Link model checkpoint as input artifact
    model_art = wandb.use_artifact(
        f'model-checkpoints/{model_name}:latest'
    )
    run.use_artifact(model_art)

    print(f'Artifact logged: {artifact.name}:{artifact.version}')
```

Linking model checkpoints as input artifacts creates an end-to-end lineage graph: model → evaluation → results — essential for compliance and reproducibility audits.

W&B Dashboard — Run Comparison View

W&B Project: llm-eval-pipeline · Runs: 8 · Metrics View

Run Name	mmlu ↑	TruthfulQA ↑	Custom QA ↑	Judge Win% ↑	Date
llama31-8B-v2-5shot	0.721	0.652	0.741	71.2%	2026-05-14
llama31-8B-v1-5shot	0.683	0.621	0.699	61.4%	2026-05-07
mistral-7B-inst-5shot	0.657	0.598	0.671	44.8%	2026-05-06

Best run highlighted in green — v2 model improves MMLU +3.8 pts and judge win rate +9.8 pts over v1.

PART E continued

GitHub Actions CI — Automated Regression Testing

GitHub Actions — eval_ci.yaml (Part 1)

YAML

```
# ci/eval_ci.yaml
name: LLM Evaluation CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  evaluate:
    runs-on: [self-hosted, gpu]    # Requires GPU runner
    timeout-minutes: 120

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python 3.11
        uses: actions/setup-python@v5
        with: {python-version: '3.11'}
```

GitHub Actions — eval_ci.yaml (Part 2)

YAML

```
- name: Install dependencies
  run: pip install -r requirements-eval.txt

- name: Download held-out dataset
  run: aws s3 cp s3://zenteiq-eval-data/held_out_qa.jsonl eval/data/
  env:
    AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
    AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }

- name: Run evaluation pipeline
  run: python eval/eval_runner.py --model $MODEL_PATH --ci
  env:
    MODEL_PATH: ${ vars.EVAL_MODEL_PATH }
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
    WANDB_API_KEY: ${ secrets.WANDB_API_KEY }

- name: Check regression thresholds
  run: python ci/check_thresholds.py --results results/ci_results.json
```

Store all API keys as GitHub Secrets — never hardcode or log them. The held-out dataset is fetched from encrypted S3 on every CI run.

CI Threshold Checker — check_thresholds.py

Python

```
# ci/check_thresholds.py
import json, sys, argparse

THRESHOLDS = {
    'mmlu/acc_norm': 0.65, # Must not drop below 65%
    'truthfulqa/acc': 0.60, # Must not drop below 60%
    'custom_qa/exact_match': 0.65, # Must not drop below 65%
    'judge/win_rate': 0.55, # Must beat baseline by >5%
}

def check(results_path: str) -> bool:
    with open(results_path) as f:
        results = json.load(f)
    failures = []
    for metric, threshold in THRESHOLDS.items():
        value = results.get(metric, 0)
        if value < threshold:
            failures.append(f'FAIL {metric}: {value:.3f} < {threshold}')
    if failures:
        print('\n'.join(failures)); sys.exit(1) # Blocks merge
    print('All thresholds passed.');
```

`sys.exit(1)` causes the GitHub Actions step to fail — blocking the PR merge. Set thresholds 2–3 pp below your current best to allow for noise.

Evaluation Orchestrator — eval_runner.py

Python

```
# eval/eval_runner.py
import wandb, json, argparse
from lm_eval import simple_evaluate
from lm_eval.models.huggingface import HFLM
from judge.judge_client import judge_pairwise

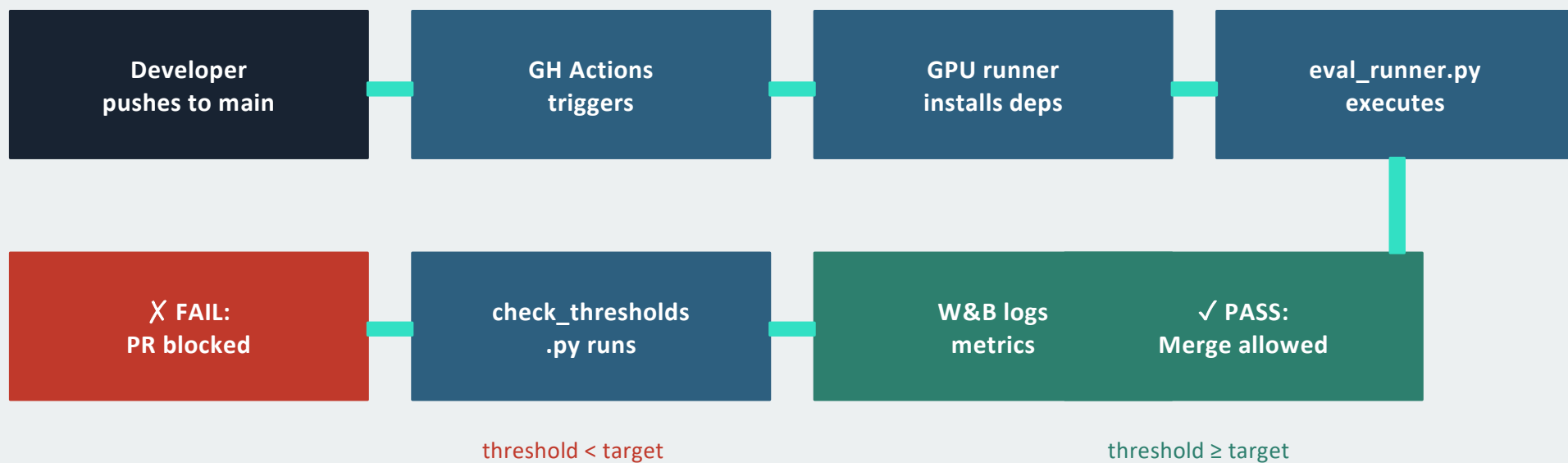
def main(model_path: str, ci: bool = False):
    run = wandb.init(project='llm-eval-pipeline', job_type='evaluation')

    # Run benchmarks
    lm = HFLM(pretrained=model_path, dtype='bfloat16', device='cuda')
    results = simple_evaluate(model=lm,
                              tasks=['mmlu', 'truthfulqa_mc2', 'domain_qa_held_out'],
                              num_fewshot=5, output_path='results/ci_results.json')

    # Log to W&B
    run.log({'mmlu/acc_norm': results['results']['mmlu']['acc_norm', None]})
    run.finish()

    if ci:
        json.dump({'mmlu/acc_norm': results['results']['mmlu']['acc_norm', None]},
                  open('results/ci_results.json', 'w'))
```

GitHub Actions CI — End-to-End Workflow



Pipeline completes in ~1.5 hours on an A100 GPU runner. With a smaller fast-check subset (500 questions), can complete in ~12 minutes.

CI Evaluation — Best Practices

Learned from running evaluation pipelines in production:

- Use a fast-check subset (200–500 questions) for PR validation; reserve full benchmark for nightly main-branch runs
- Set regression thresholds at `current_best - 2pp` — tight enough to catch real regression, loose enough to absorb evaluation noise
- Version the threshold config in Git — when you update the model and improve scores, update thresholds in the same PR
- Alert on `parse_error_rate > 5%` from the judge — this indicates a prompt engineering problem, not a model regression
- Store W&B run links in PR comments via the wandb GitHub app — reviewers can inspect every evaluation result from the PR interface
- Never evaluate the same model twice on the same held-out set with different results — if results differ, your pipeline is not deterministic

Fast-Check Subset — PR Evaluation Configuration

Python

```
# eval/configs/fast_check.py
# Subset of tasks for fast PR validation (~12 min on A100)

FAST_CHECK_CONFIG = {
    'tasks': [
        'mmlu_medical',      # 6-subject subset (not all 57)
        'truthfulqa_mc2',    # All 817 questions, fast
        'domain_qa_held_out', # Custom 200 examples
    ],
    'num_fewshot': 5,
    'batch_size': 32,       # Larger batch for speed
    'limit': 100,           # Max 100 examples per task
    'seed': 42,
}

NIGHTLY_CONFIG = {
    'tasks': ['mmlu', 'truthfulqa_mc2', 'gsm8k',
              'arc_challenge', 'domain_qa_held_out'],
    'limit': None,          # Full benchmark
}
```

Use `--limit 100` in `lm-eval` CLI for fast-check runs. This evaluates a random sample of 100 examples per task — sufficient for catching large regressions.

Building Evaluation Pipelines — Implementation Checklist

Component	Tool / Approach	Key Config	Done?
Environment	conda + requirements-eval.txt	Pinned versions, isolated env	☐
Benchmark runner	lm-eval-harness 0.4.3	YAML task config, num_fewshot	☐
Custom held-out	JSONL + ConfigurableTask class	200+ examples, 3 difficulty levels	☐
LLM Judge	judge_client.py + Anthropic API	Position bias mitigation, JSON output	☐
Metric aggregation	exact_match + F1 + BERTScore	Category breakdown, per-sample logging	☐
W&B tracking	wandb.init + log + artifact	git_hash, run groups, Tables	☐
CI pipeline	GitHub Actions + GPU runner	Threshold check, PR blocking	☐

Key Takeaways

AI-SA-IN-CD-000001

- 1 A production evaluation pipeline has 5 layers: benchmark runner (lm-eval), custom task evaluator, LLM judge, W&B tracker, and CI regression check
- 2 lm-eval-harness with YAML task configs provides one-command reproducible benchmark evaluation — always pin versions and log git hash
- 3 Custom held-out tasks are your most valuable evaluation signal — private, contamination-resistant, and directly measure your deployment use case
- 4 The LLM judge client must mitigate position bias (randomise order) and parse structured JSON output — handle parse errors explicitly
- 5 W&B Tables + artifact versioning create a full evaluation lineage: every metric traces back to the exact model checkpoint and code version
- 6 GitHub Actions CI with hard threshold checks turns evaluation from a manual process into an automated quality gate — blocking regressions before they reach production